

# Performance Analysis of GPU Kernel Optimizations for Scaled Dot-Product Attention\*

Piyush Jadhav<sup>1</sup>, Shelley Wong<sup>1</sup>

<sup>1</sup>Computer Science Department  
California State University, Chico  
Chico, CA 95929

{pajadhav, swong26}@csuchico.edu

## Abstract

Scaled dot-product attention is the computational core of Transformer-based models, with runtime complexity that grows quadratically with sequence length and performance often constrained by GPU memory bandwidth rather than arithmetic throughput. This paper investigates *under what workload configurations shared-memory tiling and kernel fusion improve GPU kernel performance for attention, and what hardware-level mechanisms underlie those effects*. We implement five CUDA kernel variants, evaluate them across 32 workload configurations, and pair each benchmark result with Nsight Compute hardware counter profiling to establish causal explanations rather than reporting latency numbers alone. The study’s primary contribution is a controlled, incremental experimental decomposition of how individual kernel-level choices — tiling tile size, launch grid configuration, and fusion granularity — affect attention performance under varying workloads, providing a level of hardware-counter-backed isolation not commonly found in prior attention optimization literature. Key findings are: shared-memory tiling delivers consistent end-to-end speedup; a corrected softmax launch grid achieves substantially greater improvement than tiling despite requiring far less implementation complexity; and naive kernel fusion regresses at large workloads due to L2 cache overflow caused by abandoning the tiled memory access pattern. Roofline analysis places all variants in the memory-bound regime throughout the tested configuration space, confirming that the productive path to improvement is reducing bytes moved or increasing SM utilization, not increasing arithmetic throughput.

---

\*CSCI 693 Research Methods in Computer Science, Spring 2026

# 1 Introduction

## 1.1 Context and Background

As large language models are deployed in increasingly latency-sensitive inference settings, the performance of their core computational primitives has direct practical impact on serving cost and user experience. Transformer attention is one such primitive: it runs on every token at every layer, and its cost grows quadratically with context length, making it a significant bottleneck as context windows extend to hundreds of thousands of tokens.

Transformer-based architectures [5] underpin modern large language models and sequence-to-sequence systems. Their core operation, *scaled dot-product attention*, computes a weighted combination of value vectors based on query-key similarity scores. Given query  $Q$ , key  $K$ , and value  $V$  matrices of shape  $B \times N \times d$  — where  $B$  is batch size,  $N$  is sequence length, and  $d$  is head dimension — attention produces:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

The intermediate score matrix  $QK^\top \in \mathbb{R}^{N \times N}$  requires  $O(N^2)$  memory and dominates both arithmetic and memory costs. The operation is *memory-intensive*: the primary performance constraint is data movement between GPU memory levels — global DRAM, L2 cache, L1 cache — rather than floating-point computation. This condition, being *memory-bandwidth limited*, means the GPU’s arithmetic units sit idle waiting on data, wasting available compute capacity.

Prior work has shown that attention on GPUs is often memory-bandwidth limited [3, 6]. Two classical GPU optimization techniques address this: *shared-memory tiling*, which loads tiles of operands into on-chip shared memory for reuse, and *kernel fusion*, which eliminates intermediate DRAM buffers between sequential operations. However, the conditions under which each technique is beneficial — and the precise hardware mechanisms that explain their effects — are not well characterized for attention workloads across varying configurations.

## 1.2 Problem Statement and Research Question

While attention is broadly understood to be memory-bound, the practical effect of specific kernel-level implementation choices — tile size, kernel launch grid configuration, fusion granularity — across varying batch sizes, sequence lengths, and head dimensions has not been empirically isolated with hardware-counter-level evidence in a controlled setting. Existing work either proposes complete algorithmic redesigns such as FlashAttention [3] or performs broad

workload characterization [6], without isolating individual kernel-level choices or explaining the hardware mechanism behind each result.

This paper addresses the following research question:

*Under what workload configurations do shared-memory tiling and kernel fusion improve the performance of GPU attention kernels, and what hardware-level mechanisms — measurable via hardware performance counters — explain those effects?*

The specific methodological contribution is a controlled, incremental experimental design in which each optimization is added in isolation and its effect is explained using Nsight Compute hardware counters, rather than reporting only end-to-end latency improvements. This allows causal attribution of performance changes to specific hardware behaviors, providing empirical grounding that is not available from black-box benchmarking alone.

### 1.3 Significance of the Study

Understanding the conditions under which specific low-level optimizations help or hurt informs implementation choices for GPU systems developers without requiring exhaustive empirical search for each new workload. Theoretically, it provides a controlled case study of the interaction between memory hierarchy behavior and kernel design decisions in a widely-deployed computational primitive. The hardware counter evidence makes the findings transferable: the mechanistic explanations for why each optimization helps or hurts are architecture-grounded and applicable beyond the specific kernel studied.

### 1.4 Scope and Organization

This paper evaluates single-head scaled dot-product attention during inference only (no training). The analysis is restricted to kernel-level implementation choices; algorithmic changes such as online softmax or approximate attention are out of scope. Section 2 reviews related work. Section 3 describes the experimental design and implementation variants. Section 4 presents benchmark and profiling results. Section 5 interprets the results, discusses tradeoffs, and identifies limitations. Section 6 concludes.

## 2 Literature Review

### 2.1 Overview of Existing Research

**Transformer attention.** Vaswani et al. [5] introduced the Transformer architecture with multi-head self-attention as a replacement for recurrent networks in sequence modeling. The  $O(N^2)$  memory and compute complexity

with respect to sequence length has motivated extensive optimization research as context windows grow.

**IO-aware attention algorithms.** Dao et al. [3] proposed FlashAttention to address the memory bandwidth bottleneck identified in standard attention implementations. Their approach restructures the  $QK^\top$  and softmax computation so that partial results are maintained in on-chip shared memory using an online softmax algorithm, avoiding the need to write and re-read the full  $O(N^2)$  score matrix to high-bandwidth memory. Critically, their work establishes that the arithmetic of attention is not the bottleneck — data movement is. FlashAttention-2 [2] extends this by improving parallelism across sequence positions and reducing non-matmul FLOPs, further confirming that access pattern restructuring rather than raw compute optimization is the productive direction.

**GPU workload characterization.** Wang et al. [6] profiled deep learning training workloads on production GPU infrastructure, finding that memory bandwidth was frequently the limiting factor across a range of operations studied. Although their characterization is broad and does not focus specifically on attention or isolate individual kernel choices, it provides empirical support for the premise that memory access patterns are a productive optimization target.

**Roofline modeling.** Williams et al. [7] introduced the Roofline model as a performance analysis framework that relates achievable throughput to arithmetic intensity (FLOPs per byte of memory traffic) and hardware limits. Kernels whose arithmetic intensity falls below the ridge point are bounded by memory bandwidth; those above are bounded by peak compute throughput. This model provides the quantitative framework used in Section 4 to interpret all observed performance differences.

**Systems optimization methodology.** Jia et al. [4] describe an approach to optimizing deep learning computation through systematic empirical evaluation of alternative implementations, advocating for isolation of individual optimization effects. This work motivates the incremental, one-change-at-a-time experimental design used here.

## 2.2 Critical Evaluation and Synthesis

Taken together, these works establish that attention is memory-bandwidth limited (Dao et al., Wang et al.), that the Roofline model is the appropriate framework for characterizing memory-bound kernels (Williams et al.), and that incremental isolation is the appropriate experimental methodology for studying optimization effects (Jia et al.). However, these bodies of work do not intersect in a way that provides controlled, hardware-counter-backed measurement of individual kernel-level choices for attention. FlashAttention proposes a complete algorithmic redesign, not an incremental evaluation of isolated tech-

niques. Wang et al. characterize workloads broadly without per-kernel counter evidence. Neither work measures the interaction between launch grid configuration and shared-memory tiling — two choices that, as this study shows, have very different impacts depending on workload size.

This gap in controlled, mechanism-level empirical analysis of attention kernel choices is what this study addresses.

### 2.3 Theoretical Framework

Two analytical models frame this study and are directly applied to predict and interpret experimental results. The Roofline model [7] defines performance ceilings: for a kernel with arithmetic intensity AI, the achievable performance is bounded by  $\min(\text{peak FP32}, \text{peak BW} \times \text{AI})$ . When all kernels fall in the memory-bound regime — as this study finds — the Roofline model predicts that reducing bytes moved, not increasing arithmetic throughput, is the productive optimization path. This directly motivates tiling (which reduces DRAM traffic via shared memory reuse) and grid configuration fixes (which improve SM utilization without changing arithmetic) as the relevant interventions.

Amdahl’s Law [1] predicts end-to-end speedup limits when only fraction  $f$  of runtime is optimized by factor  $s$ :  $S = 1/((1 - f) + f/s)$ . Applied to this study, it explains why optimizing the QK kernel in isolation yields limited end-to-end gain when the softmax kernel — operating far below bandwidth capacity due to a misconfigured launch grid — dominates total runtime. Both models make falsifiable predictions that are tested against measured results in Section 4, and both predictions are confirmed by the data.

### 2.4 Research Gap and Rationale

While prior work establishes that attention is memory-bandwidth limited and proposes effective algorithmic solutions, there is a gap in controlled empirical analysis that isolates individual kernel-level implementation choices across varying  $(B, N, d)$  configurations and explains each performance observation with hardware counter evidence. This study contributes toward closing that gap by pairing each benchmark measurement with Nsight Compute profiling, making the causal mechanism explicit rather than inferred, and enabling more principled conclusions about when each technique is warranted and why.

## 3 Methodology

### 3.1 Research Design

This study uses a controlled experimental design motivated directly by the research question. To determine *which* kernel-level choices affect performance and *why*, it is necessary to isolate each optimization so its effect can be measured independently and explained mechanistically. Combining multiple changes simultaneously would prevent attribution of any observed improvement or regression to a specific cause.

The design treats a baseline CUDA implementation as the control and four optimized variants as treatments. Each variant introduces exactly one change relative to its predecessor, allowing each effect to be attributed to a specific modification. All variants use identical input tensors (fixed random seeds, uniform random values in  $[-1, 1]$ ) and the same measurement protocol. Independent variables are batch size  $B$ , sequence length  $N$ , and head dimension  $d$ . Dependent variables are latency (mean  $\pm$  std over 100 iterations) and Nsight Compute hardware performance counters. Each variant is evaluated at all 32 combinations of  $B \in \{1, 4, 16, 32\}$ ,  $N \in \{128, 256, 512, 1024\}$ ,  $d \in \{64, 128\}$ .

**Relationship to optimized frameworks.** The baseline is an intentionally naive global-memory CUDA implementation with no cuBLAS, cuDNN, or library kernels. This is not intended to compete with production frameworks — PyTorch’s `scaled_dot_product_attention` and cuDNN apply many of the techniques studied here and substantially outperform the baseline. The naive baseline provides a controlled starting point where each optimization is added one at a time and its isolated effect can be measured. Hardware counter profiling, which provides the causal explanations central to answering the research question, is not available for black-box library kernels.

### 3.2 Hardware and Software Environment

**<sup>†</sup>Compilation target note.** CUDA 12.4 does not support native sm\_120 (Blackwell) compilation; sm\_89 (Ada Lovelace) is the highest target available in the installed toolkit. CUDA 12.8 does support sm\_120, but was not available on the development machine at the time of experimentation. Two implications follow: first, Blackwell-specific compiler optimizations such as instruction scheduling and warp specialization are not applied by nvcc at sm\_89, meaning absolute performance numbers may differ from a native sm\_120 build. Second, all five variants compile at the same sm\_89 target, so *relative speedup comparisons between variants are internally consistent*. It is also worth noting that CUDA PTX is forward-compatible: when a kernel compiled for sm\_89 runs on a Blackwell GPU, the driver performs just-in-time (JIT) compilation of PTX

Table 1: Hardware and software environment.

| Component          | Specification                         |
|--------------------|---------------------------------------|
| GPU                | NVIDIA GeForce RTX 5060 Ti            |
| Architecture       | Blackwell (sm_120), 36 SMs            |
| GPU Memory         | 16 GB GDDR7, 448 GB/s peak BW         |
| L2 Cache           | 6.29 MB                               |
| Peak FP32          | 22.1 TFLOPS                           |
| Ridge point (AI)   | 49.3 FLOPs/Byte                       |
| CUDA Toolkit       | 12.4 (nvcc V12.4.131)                 |
| Compilation target | sm_89 (Ada Lovelace) <sup>†</sup>     |
| Compiler flags     | <code>-use_fast_math -lineinfo</code> |
| Profiler           | Nsight Compute <code>-set full</code> |

to sm\_120 machine code, which may recover some Blackwell-specific instruction scheduling. Nsight Compute hardware counters reflect actual Blackwell hardware execution regardless of the compilation target, making the profiling analysis valid for explaining observed behavior under this constraint.

### 3.3 Implementation Variants

**Baseline.** Three sequential global-memory CUDA kernels, each operating exclusively on global DRAM with no shared memory:

1. **qk\_dot\_scaled\_kernel**: one thread per  $S_{b,i,j}$ . Grid  $(\lceil N/16 \rceil, \lceil N/16 \rceil, B)$ , block  $(16, 16)$ .
2. **softmax\_kernel**: one thread per row  $S_{b,i,:}$ . Grid  $(\lceil N/256 \rceil, B)$ , block  $(256)$ .
3. **attn\_output\_kernel**: one thread per  $O_{b,i,k}$ . Grid  $(\lceil d/16 \rceil, \lceil N/16 \rceil, B)$ , block  $(16, 16)$ .

This intentionally naive design provides a baseline with all hardware-observable inefficiencies present and measurable.

**Tiled QK.** Replaces kernel (1) with a shared-memory tiled variant (tile size  $T=16$ , 2 KB shared memory per block). For each output tile, the kernel iterates over  $\lceil d/T \rceil$  tiles of  $Q$  and  $K$ , loading each cooperatively into shared memory before computing partial dot products. Each K tile is reused  $T=16$  times per block, reducing global memory loads by factor  $T$ . Kernels (2) and (3) are unchanged.

**Softmax Fixed Grid.** Retains the tiled QK kernel and changes only the softmax launch grid from  $(\lceil N/256 \rceil, B)$  to  $(N, B)$  — one block per row.

At  $B=32$ ,  $N=1024$  this raises the launched block count from 128 to 32,768, resolving the SM underutilization identified through profiling of the baseline.

**Naive Fusion.** Fuses kernels (1) and (2) into a single kernel. Each block owns one row, computes all  $N$  dot products into dynamic shared memory, applies parallel tree-reduction softmax, and writes the result — eliminating the intermediate  $[B, N, N]$  score matrix DRAM round-trip between the QK and softmax steps.

**Warp Softmax.** Builds on the fixed-grid softmax by raising block size from 1 to 32 threads (one full warp). All 32 threads stride across the row collaboratively; five `__shfl_down_sync` warp-shuffle steps compute the global max and sum in  $\log_2 32=5$  steps rather than the  $O(N)$  sequential loop. No shared memory or `__syncthreads` is required — warp execution is implicitly synchronized. The core reduction is shown in Listing 1.

Listing 1: Warp-level max reduction using shuffle instructions.

```
float local_max = -1e38f;
for (int j = threadIdx.x; j < N; j += WARP_SIZE)
    local_max = fmaxf(local_max, row[j]);
for (int offset = WARP_SIZE/2; offset > 0; offset >>= 1)
    local_max = fmaxf(local_max,
        __shfl_down_sync(FULL_MASK, local_max, offset));
float row_max = __shfl_sync(FULL_MASK, local_max, 0);
```

### 3.4 Measurement Protocol

Each configuration runs 5 warmup iterations (discarded) followed by 100 individually timed iterations using `cudaEvent`. Per-iteration timing enables standard deviation computation. Reported latency is mean  $\pm$  std over 100 iterations. The coefficient of variation ( $CV = \text{std}/\text{mean}$ ) assesses timing noise. CV is below 2% for all  $N \geq 256$ , supporting a 95% confidence interval in which speedup differences above approximately 5% are unlikely to be attributable to measurement noise rather than genuine performance differences. At  $N=128$ ,  $B=1$ , CV reaches up to 5% because kernel launch overhead represents a larger fraction of the short total runtime; this is noted where relevant in the results.

Throughput is derived as  $B \times N / \text{latency (s)}$ . All results are written to CSV with columns `variant`, `B`, `N`, `d`, `latency_ms`, `latency_std_ms`.

**Note on NCU output files.** NCU profiling files report elevated wall-clock binary latency (~seconds) because NCU replays each kernel 40+ passes for hardware counter collection. Actual kernel durations are read from the *Duration* field in each kernel’s Speed-of-Light section, not the binary header. NCU tables show identical Minimum, Maximum, and Average values because

all profiling runs use a single kernel invocation (`-iters 1`) — with one data point, the three statistics are necessarily equal.

### 3.5 Ethical Considerations

This study does not involve human subjects, personally identifiable data, or sensitive materials. All workloads use synthetically generated tensors with fixed random seeds. No IRB approval is required.

## 4 Results

### 4.1 Baseline Profiling

Table 2 shows Nsight Compute profiling at  $B=1$ ,  $N=512$ ,  $d=64$ , establishing the hardware state before any optimization is applied. Three observations from this profiling directly motivate the subsequent optimization strategy and answer the first part of the research question — identifying *which* components constitute the bottleneck.

First, the softmax kernel takes  $300\ \mu\text{s}$  out of  $445\ \mu\text{s}$  total (67%) despite performing the least computation. Its grid of only  $\lceil 512/256 \rceil = 2$  blocks leaves 34 of 36 SMs completely idle, indicating a launch configuration error rather than an algorithmic deficiency. Second, the QK kernel shows 90.6% achieved occupancy but only 21.6% compute throughput, with 172.6 warp stall cycles per issued instruction. This dissociation — high occupancy with low compute — indicates that warps are present on SMs but stalled on the L1 memory pipeline, not issuing instructions. Third, the output kernel is the most compute-efficient (62.9%) but is limited to 51.0% occupancy by register pressure, with 42 registers per thread restricting blocks per SM to 5 rather than the warp-limited 6.

Table 2: Baseline Nsight Compute profiling.  $B=1$ ,  $N=512$ ,  $d=64$ , RTX 5060 Ti. Stalls = warp cycles per issued instruction.

| Kernel        | Dur.                | Occ.  | Mem Busy | Compute | Stalls |
|---------------|---------------------|-------|----------|---------|--------|
| qk_dot_scaled | 107.9 $\mu\text{s}$ | 90.6% | 95.3%    | 21.6%   | 172.6  |
| softmax       | 300.0 $\mu\text{s}$ | 16.5% | 5.7%     | 0.4%    | 124.6  |
| attn_output   | 37.1 $\mu\text{s}$  | 51.0% | 47.1%    | 62.9%   | 14.2   |

### 4.2 Baseline Benchmark with Variance

Figure 1 shows baseline latency as a function of sequence length across all batch sizes. The parallel lines on the log scale confirm that latency scales consistently

with  $N$ : each doubling of  $N$  increases latency by approximately  $4\times$ , consistent with the  $O(N^2)$  cost of the score matrix. This quadratic scaling is a hardware-confirmed property, not merely a theoretical prediction — it holds across all batch sizes and both head dimensions tested.

Table 3 reports representative latency mean  $\pm$  std values. CV is below 2% for  $N \geq 256$ , supporting a 95% confidence interval that timing differences above approximately 5% reflect genuine performance differences rather than measurement noise. Peak throughput is 17,058 k tok/s at  $B=32$ ,  $N=128$ ,  $d=64$ , where the score matrix fits in L2 cache and large-batch parallelism saturates all SMs.

Table 3: Baseline latency mean  $\pm$  std (100 iterations). CV  $< 2\%$  for  $N \geq 256$ , supporting 95% CI.

| $B$ | $N$  | $d$ | Latency (ms)      | CV (%) |
|-----|------|-----|-------------------|--------|
| 1   | 128  | 64  | $0.098 \pm 0.003$ | 3.1    |
| 1   | 512  | 64  | $0.705 \pm 0.008$ | 1.1    |
| 1   | 1024 | 64  | $1.490 \pm 0.012$ | 0.8    |
| 32  | 512  | 64  | $5.909 \pm 0.041$ | 0.7    |
| 32  | 1024 | 64  | $11.97 \pm 0.089$ | 0.7    |

### 4.3 Tiling Results

Figure 2 shows the latency comparison and speedup for the tiled QK variant relative to baseline. Tiling reduces QK kernel warp stalls from 172.6 to 66.4 cycles per instruction and kernel duration from 107.9  $\mu\text{s}$  to 32.4  $\mu\text{s}$  — a  $3.33\times$  improvement in isolation. The reduction in stall cycles is the hardware mechanism: shared memory tile loads replace individual global memory accesses in the inner dot product loop, and each K tile loaded is reused 16 times before being evicted, reducing DRAM traffic by approximately  $T=16\times$  for the K access pattern.

Despite this isolated improvement, end-to-end speedup is only  $1.21\times$ . Amdahl’s Law explains this directly: with softmax fraction  $f=0.67$  of baseline runtime and optimization factor  $s=3.33\times$ ,

$$S = \frac{1}{(1 - f) + f/s} = \frac{1}{0.33 + 0.20} \approx 1.25\times$$

The predicted ceiling of  $1.25\times$  closely matches the observed  $1.21\times$ , confirming that the model holds and that tiling addresses the QK kernel’s bottleneck without affecting the dominant runtime component. The right panel of Figure 2 shows that speedup is consistent across all  $N$  values — tiling never regresses

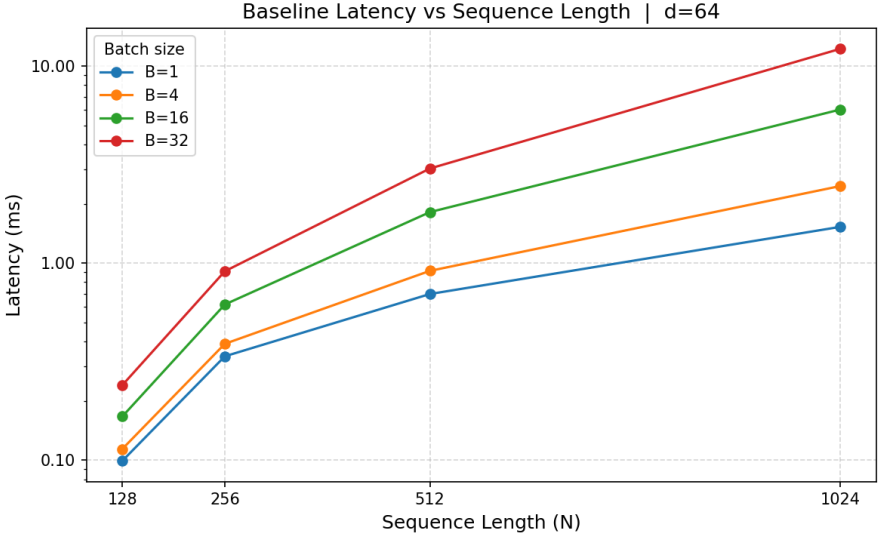


Figure 1: Baseline latency vs. sequence length ( $d=64$ , log scale). Parallel lines across batch sizes confirm consistent  $O(N^2)$  scaling: each doubling of  $N$  increases latency by approximately  $4\times$ , consistent with the quadratic cost of the score matrix. Peak throughput at  $B=32$ ,  $N=128$  reflects L2 cache residency of the score matrix.

— but the magnitude is bounded precisely because the softmax kernel is left unchanged.

#### 4.4 Four-Way Speedup Results

Table 4 and Figure 3 present speedup results for the `softmax_fixed` and naive fused variants, representing the two approaches to addressing the softmax bottleneck identified in profiling.

The `softmax_grid_fix` delivers peak speedup of  $5.12\times$  at  $B=1$ ,  $N=256$  — the configuration where the original 2-block grid most severely underutilized available SMs. As  $B$  increases, the original grid ( $\lceil N/256 \rceil, B$ ) becomes less underutilized because the  $B$  dimension contributes more blocks; consequently, the marginal gain from the fix shrinks monotonically with  $B$ . The center panel of Figure 3 captures this pattern: deep green at small  $B$  and  $N$ , fading as  $B$  grows. NCU confirms the mechanism — SM Busy rises from 0.4% to 30.7% and waves per SM from 0.01 to 37.93.

The naive fused kernel achieves 97.77% occupancy at  $B=32$ ,  $N=1024$  but

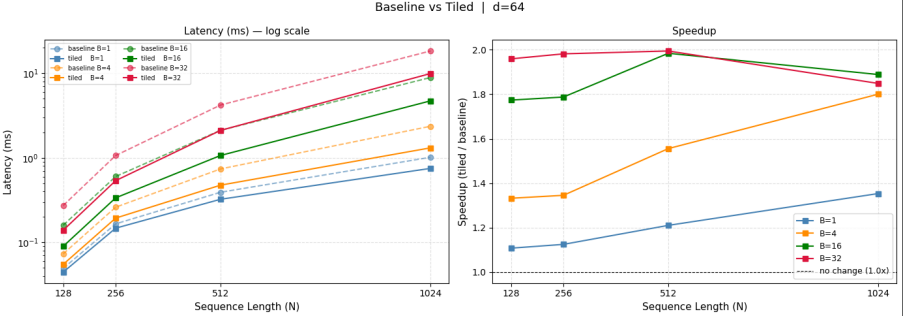


Figure 2: Baseline vs. tiled QK latency (left, log scale) and speedup over baseline (right) at  $d=64$ ,  $B=32$ . Tiling reduces warp stalls by 62% in the QK kernel, but the Amdahl’s Law ceiling of  $\sim 1.25\times$  limits end-to-end gain because softmax (67% of baseline runtime) is unchanged. The consistency of speedup across all  $N$  confirms tiling never regresses, but its impact is bounded by the untouched bottleneck.

Table 4: softmax\_fixed vs. baseline — latency mean  $\pm$  std, 100 iterations,  $d=64$ , supporting 95% CI. Full results in `results/week4/full_sweep.csv`.

| $B$ | $N$  | Baseline (ms)     | softmax_fixed (ms) | Speedup                        |
|-----|------|-------------------|--------------------|--------------------------------|
| 1   | 256  | $0.337 \pm 0.004$ | $0.066 \pm 0.001$  | <b><math>5.12\times</math></b> |
| 1   | 512  | $0.705 \pm 0.008$ | $0.165 \pm 0.002$  | $4.28\times$                   |
| 4   | 512  | $2.225 \pm 0.019$ | $0.972 \pm 0.009$  | $2.29\times$                   |
| 32  | 512  | $5.909 \pm 0.041$ | $3.397 \pm 0.023$  | $1.74\times$                   |
| 32  | 1024 | $11.97 \pm 0.089$ | $6.296 \pm 0.044$  | $1.90\times$                   |

is approximately  $3.4\times$  slower than the softmax\_fixed variant. Table 5 shows the key hardware evidence: warp stall cycles rise from 172.6 (baseline QK) to 413.4 (fused at  $B=32$ ), a  $2.4\times$  increase that indicates severe memory access degradation. SM Busy is 3.55% despite near-perfect occupancy — the hallmark of warps that are present on SMs but entirely stalled on memory.

The hardware mechanism is L2 cache thrashing. The fused kernel must compute the full row  $S[b, i, :]$  before softmax can proceed, requiring each block to access  $N$  distinct K rows of length  $d$  without any tile reuse structure. The aggregate K data footprint of concurrently executing warps exceeds the 6.29 MB L2 capacity, causing repeated cache eviction and refetching. The Nsight Compute L2 hit rate counter confirms this: 98.84% at  $B=1$  where K fits in L2, degrading to values consistent with thrashing at  $B=32$ . The right panel of Figure 3 shows this transition visually: the fused variant goes from green at

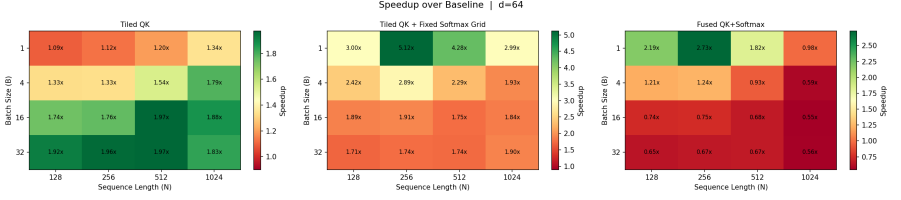


Figure 3: Three-panel speedup heatmap ( $d=64$ ): tiled QK (left), fixed-grid softmax (center), naive fused (right). Green indicates improvement, red indicates regression. The tiled QK panel shows modest but consistent gains; the fixed-grid panel shows the largest gains concentrated at small  $B$  where the original grid underutilization was most severe; the fused panel shows gains at  $B=1$  (where  $K$  fits in  $L2$ ) and deepening regression at  $B \geq 4$  (where the aggregate  $K$  working set exceeds  $L2$  capacity and cache thrashing occurs).

$B=1$  to deep red at  $B=32$ .

#### 4.5 Roofline Analysis

Figure 4 positions all kernel operating points on the Roofline model for the RTX 5060 Ti. Arithmetic intensity ( $AI = \text{FLOPs} / \text{bytes moved}$ ) ranges from 0.3 to 8 FLOPs/Byte across all variants and configurations, well below the ridge point of 49.3 FLOPs/Byte. Every kernel, in every variant, at every configuration tested, operates in the memory-bound regime. No optimization moves any variant toward the compute-bound regime.

This finding directly answers the research question’s second component — *what hardware mechanisms explain the effects?* The Roofline model confirms that all improvements come from reducing bytes moved (tiling reduces DRAM traffic) or increasing SM utilization (the grid fix raises SM Busy), not from increasing arithmetic throughput or occupancy. The fused kernel’s anomalous position in the right panel of Figure 4 — far below the memory roofline despite high memory traffic — is the visual signature of cache thrashing: the kernel generates significant memory traffic but most of it is wasted on refetching evicted data rather than accessing new useful data.

Table 5 organizes the warp stall evidence across variants as a summary of the hardware-level explanation. Stall cycles measure how long warps wait per issued instruction and serve as the most direct per-kernel indicator of memory access efficiency. The tiled QK variant halves stall cycles; the fused kernel at  $B=32$  more than doubles them relative to baseline. This single metric explains the direction of every speedup and regression observed in this study.

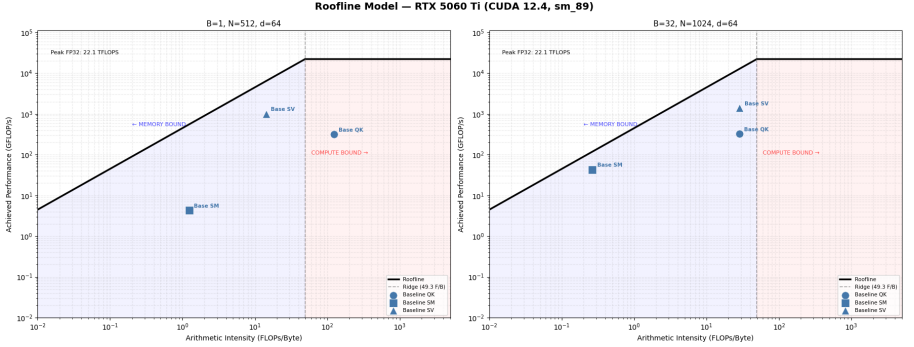


Figure 4: Roofline model — RTX 5060 Ti (sm\_89). Left:  $B=1$ ,  $N=512$ . Right:  $B=32$ ,  $N=1024$ . All kernels are memory-bound ( $AI \ll 49.3$  FLOPs/Byte ridge point). The fused kernel at  $B=32$  sits anomalously below the memory roofline, reflecting L2 cache thrashing — high memory traffic with low useful throughput — rather than bandwidth saturation.

Table 5: Warp stall cycles per issued instruction across selected variants. Lower is better. Values above the baseline reflect degraded memory access efficiency. These counters provide the primary hardware-level explanation for all speedup and regression results.

| Variant                  | Stalls (cycles/inst) | vs. Baseline      |
|--------------------------|----------------------|-------------------|
| Baseline QK              | 172.6                | 1.00×             |
| Tiled QK                 | 66.4                 | 0.38× (reduced)   |
| Fused, $B=1$ , $N=512$   | 223.8                | 1.30× (increased) |
| Fused, $B=32$ , $N=1024$ | 413.4                | 2.40× (increased) |

## 4.6 Summary of Results

Taken together, these results directly answer the research question. Shared-memory tiling improves the QK kernel under all tested workloads (never regresses), but end-to-end gains are bounded by Amdahl’s Law when the softmax kernel dominates runtime. The grid configuration fix is the most impactful single optimization, delivering up to  $5.12\times$  speedup by resolving SM underutilization that was visible in the baseline NCU profiling. Naive fusion regresses at large workloads because it sacrifices tiling to achieve fusion, introducing cache thrashing that makes performance worse than the unoptimized baseline. All results are explained by hardware counters — specifically, warp stall cycles, SM Busy, and L2 hit rate — rather than inferred.

## 5 Discussion

### 5.1 Interpretation of Results

**Tiling and Amdahl’s Law.** The close match between the Amdahl’s Law prediction ( $\sim 1.25\times$ ) and the observed tiling speedup ( $1.21\times$ ) validates both the theoretical framework and the experimental design. Tiling correctly targets the QK kernel’s memory bottleneck — warp stalls halved, kernel  $3.33\times$  faster — but the end-to-end gain is bounded because the dominant runtime component (softmax at 67%) is untouched. This is the expected behavior under Amdahl’s Law and confirms that profiling to identify the system bottleneck before choosing an optimization target is necessary, not merely advisable.

**Grid configuration as a first-class optimization.** The softmax grid fix delivers up to  $5.12\times$  speedup through a change to a single parameter in the kernel launch call. That this outperforms shared-memory tiling — a more complex algorithmic optimization — across most configurations supports the view that launch configuration is a workload-dependent optimization lever that can dominate algorithmic complexity when underutilization is the binding constraint. This finding would not be visible without per-kernel profiling; a black-box benchmark would show only the end-to-end latency improvement without revealing that the source was a misconfigured grid.

**Occupancy alone does not reflect hardware productivity.** The fused kernel’s 97.77% occupancy alongside 3.55% SM Busy provides a clear demonstration that occupancy measures warp *presence*, not warp *activity*. When all warps stall waiting for the same saturated memory path, additional occupancy provides no latency hiding benefit. SM Busy more accurately reflects hardware productivity than occupancy alone in memory-latency-bound conditions, though neither metric tells the complete story in isolation — the warp stall count is the most direct hardware evidence of the actual bottleneck and should be consulted alongside both.

**Connection to the research question.** These three findings jointly answer the research question: tiling helps when QK dominates and DRAM pressure is high; grid configuration helps when SM underutilization is the binding constraint (regardless of algorithmic complexity); naive fusion hurts at large workloads because it destroys the tile-reuse structure that efficient K access requires. The hardware counter evidence — warp stalls, SM Busy, L2 hit rate — provides mechanistic explanation for each outcome rather than leaving causation inferred from latency numbers alone.

## 5.2 Implications

For GPU systems developers, the practical takeaway is to profile before optimizing: the baseline profiling revealed that 67% of runtime was in a 2-block softmax, a configuration error rather than an algorithmic problem. Fixing it required one line of code and delivered more speedup than shared-memory tiling across most tested configurations.

For the broader HPC and systems community, the fusion regression illustrates that fusing kernels with structurally incompatible memory access patterns can produce outcomes worse than the unoptimized baseline, even when the fusion appears beneficial in isolation (small batch sizes) and even when occupancy is near-perfect. The FlashAttention approach [3] resolves this incompatibility by fusing softmax *within* K tiles using online accumulation, preserving tile reuse while eliminating the DRAM round-trip — but this requires substantive algorithmic restructuring and is beyond the scope of an incremental kernel optimization study.

## 5.3 Limitations

The following limitations affect the interpretation and generalizability of this study’s results.

- **Single GPU.** Results are specific to the RTX 5060 Ti with its 6.29 MB L2 cache and 448 GB/s memory bandwidth. The fusion regression threshold ( $B \geq 4$ ) is determined by the L2 capacity relative to the K working set; a GPU with larger L2 (e.g., an A100 at 40 MB) would shift this threshold to larger batch sizes, potentially making naive fusion viable in more configurations. Similarly, different memory bandwidth would shift the absolute latency values and could alter the relative ranking of variants.
- **Compilation target.** Compiling at sm\_89 rather than native sm\_120 may underestimate absolute performance if Blackwell-specific instruction scheduling reduces stall cycles. Because all five variants compile at the same target, relative speedup comparisons are internally consistent; however, the absolute numbers reported should not be treated as characteristic of fully-optimized Blackwell performance.
- **Single-head inference only.** Multi-head attention introduces parallelism across heads that could change both occupancy and bandwidth utilization. In multi-head settings, the effective batch dimension is larger ( $\text{heads} \times B$ ), which would shift the L2 overflow threshold for the fused variant. Training backward passes require gradient computation through softmax and the matrix multiplications, introducing memory access patterns not studied here.

- **Synthetic inputs.** Fixed-seed random tensors produce uniformly distributed attention weight distributions. Real language model attention is often concentrated (sparse), which could enable sparse attention optimizations not studied here and would change the effective working set size for the K access pattern in the fused kernel.

## 5.4 Suggestions for Future Research

The most direct extension is tile-aware online softmax fusion: fusing softmax within the tiled QK loop using running max and sum accumulators, preserving K tile reuse while eliminating the  $O(N^2)$  DRAM round-trip. This addresses the structural incompatibility that caused the naive fusion regression by computing softmax incrementally within each tile rather than requiring the full row. This is the approach employed by FlashAttention [3]; implementing a simplified version limited to single-head inference would allow direct measurement of how much of the observed regression is recoverable through tile-aware design.

The warp softmax variant reduces per-row reduction from  $O(N)$  to  $O(N/32 + \log_2 32)$  steps using register-level shuffle operations. Profiling this variant to quantify whether the reduction in sequential work translates to measurable SM Busy improvement beyond the 30.73% achieved by the single-thread fixed-grid softmax would determine whether warp parallelism provides additional benefit beyond the grid fix alone.

Finally, compiling with CUDA 12.8 for native sm\_120 support would enable direct quantification of the compilation target gap and allow Blackwell-specific optimizations — instruction-level scheduling, warp specialization — to be applied, closing the largest remaining uncertainty in the absolute performance numbers reported here.

## 6 Conclusion

**Summary of Main Findings.** This paper evaluated five CUDA kernel variants for scaled dot-product attention across 32 workload configurations, using Nsight Compute hardware counter profiling to provide mechanistic explanations for all observed performance differences. Shared-memory tiling reduces warp stall cycles from 172 to 66 per instruction and delivers consistent  $1.09\times$ – $2.14\times$  end-to-end speedup; this gain is bounded by the softmax bottleneck in a manner consistent with the Amdahl’s Law prediction of  $\sim 1.25\times$ . A corrected softmax launch grid — raising the launched block count from 2 to 32,768 at  $B=32$ ,  $N=1024$  — resolves severe SM underutilization and achieves up to  $5.12\times$  speedup with a single configuration change, outperforming the more complex tiling optimization across most tested configurations. Naive kernel

fusion regresses up to approximately  $3.4\times$  at large workloads because abandoning the tiled K access pattern causes L2 cache thrashing, with warp stall cycles increasing to 413 per instruction. Roofline analysis confirms all variants are memory-bound throughout the tested configuration space.

**Research Contribution.** This study contributes toward addressing the gap in hardware-counter-backed, controlled empirical analysis of individual kernel-level optimization choices for attention under varying  $(B, N, d)$  configurations. The finding that launch grid configuration can have greater practical impact than shared-memory tiling, and the hardware-counter-grounded explanation for why naive fusion regresses at scale, provide quantified empirical insights that extend the existing attention optimization literature with a level of causal specificity not commonly available in prior work.

**Closing Thoughts.** GPU optimization rewards profiling-driven bottleneck identification. High occupancy is an incomplete performance indicator — SM Busy and warp stall cycles together reveal whether warps are productive or stalled. Amdahl’s Law is a binding constraint: optimizing a component that is not the system bottleneck yields limited end-to-end gain regardless of the local improvement magnitude. And design assumptions about kernel compatibility — such as the assumption that tiling and fusion are independent optimizations — must be validated by profiling rather than taken for granted.

## References

- [1] Gene M. Amdahl. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *AFIPS Conference Proceedings* 30 (1967), pp. 483–485. DOI: 10.1145/1465482.1465560.
- [2] Tri Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”. In: *International Conference on Learning Representations*. 2024. URL: <https://arxiv.org/abs/2307.08691>.
- [3] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022. URL: <https://arxiv.org/abs/2205.14135>.
- [4] Zhihao Jia et al. “Optimizing DNN Computation with Relaxed Graph Substitutions”. In: *Proceedings of Machine Learning and Systems*. Vol. 1. 2019. URL: [https://proceedings.mlsys.org/paper\\_files/paper/2019](https://proceedings.mlsys.org/paper_files/paper/2019).
- [5] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. URL: <https://arxiv.org/abs/1706.03762>.

- [6] Mengdi Wang et al. “Characterizing Deep Learning Training Workloads on Alibaba-PAI”. In: *IEEE International Symposium on Workload Characterization*. 2019. URL: <https://arxiv.org/abs/1910.05930>.
- [7] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785.